Web Application Vulnerability Scanner

Project Documentation Report | Cyber Security | Elevate Labs Internship | 2025

Project Title	Web Application Vulnerability Scanner
Developer	Nithish Nidumolu
Domain	Cyber Security
Type	Security Tool — Python + Flask Web Application
Frontend	HTML, CSS, Jinja2 Templates
Backend	Python 3, Flask Framework
Libraries	Requests, BeautifulSoup4
Deployment	Replit Cloud Platform
Live URL	https://web-scan--nithishnidumolu.replit.app
Internship	Elevate Labs — Cyber Security Domain
Year	2025

DESCRIPTION

The Web Application Vulnerability Scanner is a Python-based security tool designed to automatically detect common web application vulnerabilities. It targets web forms and URL parameters to identify two of the most critical vulnerability classes — Cross-Site Scripting (XSS) and SQL Injection (SQLi) — as listed in the OWASP Top 10. The tool crawls HTML forms on a target URL, injects predefined attack payloads, and analyzes server responses to identify vulnerable entry points.

A Flask-based web interface hosted live on Replit allows users to enter any target URL and view scan results directly in the browser. Upon completion, the scanner generates a structured vulnerability report listing all findings with severity levels, payloads used, and affected URLs. This project was developed as part of the Elevate Labs Cyber Security Internship.

SYSTEM REQUIREMENTS :

To ensure smooth development, deployment, and usage of the Web Application Vulnerability Scanner, certain system prerequisites must be met. These requirements are categorized into

software, setup, and hardware specifications, all of which contribute to building a robust and deployable security tool.

1. Software Requirements

These are the essential tools and platforms required to develop, test, and run the application efficiently.

- **Operating System:** Windows 10/11, macOS, or Linux — Supports cross-platform development and testing.
- **Python (v3.8 or above):** Core language for both the scanning engine and Flask web server. □ Download Python: https://python.org/
- **pip (v21 or above):** Python package manager required to install Flask, Requests, and BeautifulSoup4.
- **Flask:** A lightweight Python web framework for building the web interface and handling API routing.
- **Requests:** Python HTTP library used to send GET/POST requests to target URLs with injected payloads.
- **BeautifulSoup4:** HTML parser — extracts form elements, input fields, and action URLs from web pages.
- **Browser:** Google Chrome / Firefox (latest version) — For rendering and testing the UI in real-time.
- **Visual Studio Code:** Preferred code editor with built-in Git and terminal support.
- **Git & GitHub:** For version control and submitting the project to the repository.
- **Replit:** Cloud platform used to host and deploy the Flask application publicly online.
- **Postman:** Optional tool for testing API endpoints during development.

2. Hardware Requirements

Describes the minimum and recommended specifications needed to support the development and usage of the application.

- **Processor:** Intel Core i3 (7th Gen or above) / AMD Ryzen 3 or better — Ensures fast compilation and multitasking during development.
- **RAM:** Minimum 4 GB (8 GB recommended) — For handling the development server, IDE, and browser testing simultaneously.
- **Storage:** At least 500 MB free space — Required for Python, pip package installations, and project files.
- **Display:** 1366x768 or higher — Recommended for optimal coding experience and application layout visualization.
- **Internet:** Stable broadband connection — Required for deploying on Replit and testing against external target URLs.

3. Setup Commands

Install all required Python libraries using pip:

```
pip install flask requests beautifulsoup4
```

Run the Flask web application locally:

```
python app.py
```

The application will start on: http://localhost:5000

SCENARIO-BASED CASE STUDY

Scenario: Arjun, a junior web developer at a software company, has just finished building a customer login portal. Before going live, his security team asks him to run a quick vulnerability check on the application using the Web Application Vulnerability Scanner.

1. Accessing the Scanner :

- ➤ Arjun opens his browser and navigates to https://web-scan--nithishnidumolu.replit.app
- ➤ He sees the clean scanner interface with a URL input field and a "Start Scan" button.
- ➤ He enters the target URL of his local test application: http://testphp.vulnweb.com

2. Initiating the Scan :

- ➤ Arjun clicks the "Start Scan" button — the Flask backend receives the URL via a POST request.
- ➤ The scanner fetches the target page and uses BeautifulSoup4 to extract all HTML form elements.
- ➤ It identifies 3 forms: a login form, a search form, and a registration form.

3. XSS Detection :

- ➤ The scanner injects XSS payloads such as <script>alert('XSS')</script> into each input field.
- ➤ The search form reflects the payload back in the HTML response without sanitization.
- ➤ The scanner flags this as a HIGH severity XSS vulnerability and records the payload and URL.

4. SQL Injection Detection :

- ➤ The scanner injects SQLi payloads like ' OR 1=1-- into the login form fields.
- ➤ The server returns a response containing 'mysql_fetch_array()' — a database error string.
- ➤ The scanner flags the login form as CRITICAL severity SQL Injection vulnerable.

5. Viewing Results :

- ➤ Arjun sees the scan results rendered on the web page with color-coded severity badges.
- ➤ Each finding shows the vulnerability type, affected URL, payload used, and severity level.

- ➤ The scanner also saves a structured report to scan_report.txt for documentation.

6. Taking Action :

- ➤ Arjun shares the scan_report.txt with his development team.
- ➤ The team fixes the XSS issue by encoding output using html.escape() in Python.
- ➤ The SQLi is fixed by replacing raw SQL queries with parameterized queries using placeholders.
- ➤ A rescan confirms both vulnerabilities are resolved before the portal goes live.

PROJECT ARCHITECTURE

TECHNICAL ARCHITECTURE

The Web Application Vulnerability Scanner follows a client-server architecture where the Flask framework serves both the frontend interface and the backend scanning logic. The frontend is a simple HTML/CSS page rendered via Jinja2 templates. The backend is a Python-Flask application that handles incoming scan requests, runs the vulnerability detection engine, and returns results to the browser.

The frontend incorporates the Jinja2 templating engine to dynamically render scan results. Flask handles all routing and communication between the user interface and the scanning engine. For data storage, findings are saved to a local scan_report.txt file with timestamps, payload details, and severity ratings.

Together, the frontend templates, Flask routing layer, scanning engine (scanner.py), and file-based report system form a comprehensive technical architecture for this security tool. This architecture enables accurate vulnerability detection, efficient data exchange, and seamless deployment on the Replit cloud platform.

ARCHITECTURE TABLE

Layer	Technology	Responsibility
Presentation	HTML, CSS, Jinja2	User interface — URL input form and results display
Application	Python, Flask	Route handling, scan orchestration, report generation
Scanning Engine	Requests, BS4	Form extraction, payload injection, response analysis
Report Layer	Python file I/O	Saves findings to scan_report.txt with timestamps
Deployment	Replit Cloud	Public hosting accessible via web browser

FEATURES :

1. XSS (CROSS-SITE SCRIPTING) DETECTION

- **PAYLOAD INJECTION:** The scanner automatically injects multiple XSS payloads into all discovered form input fields on the target page.
- **REFLECTION ANALYSIS:** Responses are analyzed to check if the payload is reflected in the HTML — confirming a Reflected XSS vulnerability.

2. SQL INJECTION DETECTION

- **PAYLOAD INJECTION:** SQLi payloads such as ' OR 1=1-- are submitted via all form inputs on the target application.
- **ERROR KEYWORD MATCHING:** Server responses are scanned for database error strings like 'mysql', 'syntax', 'ORA-' to confirm SQLi vulnerability.

3. AUTOMATIC FORM CRAWLING

- **HTML PARSING:** Uses BeautifulSoup4 to automatically discover and extract all form elements, input fields, and action URLs from the target page.
- **METHOD DETECTION:** Correctly handles both GET and POST form methods for accurate payload delivery.

4. FLASK WEB INTERFACE

- **URL INPUT FORM:** Clean web interface where users enter any target URL and trigger the scan with a single button click.
- **RESULTS DISPLAY:** Scan findings are rendered dynamically on the results page with color-coded severity badges.

5. REPORT GENERATION

- **AUTO-SAVED REPORT:** Every scan automatically generates scan_report.txt with full vulnerability details, payloads, and timestamps.
- **SEVERITY RATINGS:** Each finding is categorized as HIGH (XSS) or CRITICAL (SQLi) for easy prioritization.

ROLES AND RESPONSIBILITIES :

1. SECURITY TESTER / INTERN

Example: Arjun, a developer who wants to test his login portal before launch, opens the Web Application Vulnerability Scanner. He registers the target URL and uses the scan button to

identify vulnerabilities. Once results are displayed, he takes the scan_report.txt to his team for remediation.

2. RUNNING THE INITIAL SCAN

Example: Arjun navigates to https://web-scan--nithishnidumolu.replit.app, enters http://testphp.vulnweb.com as the target, and clicks "Start Scan". The tool automatically crawls the page and extracts 3 forms for testing.

3. REVIEWING XSS FINDINGS

Example: The scanner identifies the search form as XSS-vulnerable. Arjun sees the HIGH severity badge, the reflected payload, and the affected URL — giving him exactly what he needs to fix the issue.

4. REVIEWING SQL INJECTION FINDINGS

Example: The login form is flagged as CRITICAL for SQL Injection. The report shows the payload ' OR 1=1-- triggered a MySQL error in the server response, confirming the vulnerability.

5. EXPORTING AND SHARING THE REPORT

Example: Arjun opens scan_report.txt and shares it with his senior developer. The structured report lists every finding with type, severity, URL, and payload — ready for developer action.

6. REMEDIATION VERIFICATION

Example: After the development team applies fixes (output encoding for XSS, parameterized queries for SQLi), Arjun runs the scanner again. The second scan returns clean — no vulnerabilities detected.

7. DEPLOYMENT AND SUBMISSION

Example: Arjun submits the project to his GitHub repository with the scanner code, README, and report. The live Replit deployment is shared with Elevate Labs as proof of a working, deployed security tool.

USER FLOW :

1. Security Tester / Developer:

- **Role:** Use the scanner to identify vulnerabilities in web apps before deployment.

Flow:

1. Accessing the Scanner:

- Open the browser and navigate to https://web-scan--nithishnidumolu.replit.app
- The home page loads with a URL input field and Start Scan button.

2. Running a Scan:

- Enter the target URL in the input field.
- Click "Start Scan" — the backend processes the URL and runs XSS + SQLi detection.

3. Viewing Results:

- Results page shows all findings with type, severity, affected URL, and payload.
- Color-coded severity badges make it easy to prioritize fixes.

4. Downloading the Report:

- scan_report.txt is auto-generated after every scan.
- Share with development team for remediation tracking.

2. Developer (Remediation):

- **Role:** Receive the scan report and fix identified vulnerabilities.

Flow:

1. Review Report:

- Open scan_report.txt and review all findings by severity.

2. Fix XSS:

- Apply output encoding using html.escape() or templating auto-escaping.

3. Fix SQLi:

- Replace raw SQL queries with parameterized queries or ORM.

4. Verify Fix:

- Re-run the scanner to confirm all vulnerabilities are resolved.

SCANNER ARCHITECTURE PATTERN :

The Web Application Vulnerability Scanner backend follows a modular pattern separating the web layer (Flask routes), the logic layer (scanner functions), and the output layer (report generation). This separation allows for modularity, easier maintenance, and scalability.

Flask Routes Layer (Web Layer)

The routes layer is responsible for handling all HTTP requests. It receives the target URL from the user, delegates processing to the scanning engine, and returns results to the template for rendering.

Scanning Engine Layer (Logic Layer)

The scanning engine (scanner.py) acts as the core logic layer. It receives the target URL, fetches the HTML content, extracts forms, injects payloads, and analyzes responses. All vulnerability detection logic lives here.

Report / Output Layer

The report generator creates a structured text file (scan_report.txt) with all findings. It is decoupled from the scanning logic, making it easy to replace with a database or PDF export in future versions.

Advantages of This Architecture:

- **Separation of Concerns:** Each layer has a clearly defined responsibility, improving readability and maintainability.
- **Scalability:** New vulnerability checks can be added easily by extending scanner.py.
- **Reusability:** The scanning functions can be reused via CLI or API without the Flask layer.
- **Testability:** Each layer can be tested independently — routes, scanner logic, and report output.
- **Portability:** The tool can be deployed on any cloud platform (Replit, Heroku, AWS) with minimal changes.

PROJECT SETUP AND CONFIGURATION :

Creating the project folder

1. Create a new folder named web-vuln-scanner.
2. Inside that folder create two files: app.py and scanner.py.
3. Create a templates/ folder with index.html and results.html.
4. Create a static/ folder with style.css.
5. Open the folder in VS Code.

Python environment setup

- Open the project folder in the VS Code terminal.
- Create a virtual environment (optional but recommended):

```
python -m venv venv
```

- Activate the virtual environment:

```
venv\Scripts\activate        # Windows source venv/bin/activate     # macOS / Linux
```

- Install all dependencies:

```
pip install flask requests beautifulsoup4
```

Flask app setup

- Create app.py and import Flask.
- Define routes: / for the home page and /scan for the scanning endpoint.
- Set host='0.0.0.0' and port=5000 for Replit compatibility.
- Run the development server:

```
python app.py
```

BACKEND DEVELOPMENT:

https://web-scan--nithishnidumolu.replit.app

Set Up Project Structure:

- Create app.py as the main Flask entry point and scanner.py as the core scanning engine.
- Install necessary dependencies: Flask, Requests, and BeautifulSoup4.

Create Flask Web Server:

- Set up a Flask server to handle HTTP requests and serve the web interface.
- Configure the home route (/) and scan route (/scan) with appropriate HTTP methods.

Define Scanning Routes:

- Create the /scan POST route to receive the target URL from the user input form.
- Implement route handler to call scan_xss() and scan_sqli() and pass results to the template.

Implement Scanning Functions:

- Build get_forms(url) to fetch and extract all HTML forms from the target page.
- Build get_form_details(form) to extract action URL, method, and input field names.
- Implement scan_xss(url) to inject XSS payloads and check for reflection.
- Implement scan_sqli(url) to inject SQLi payloads and check for database error strings.

Report Generation:

- Implement generate_report(url, findings) to save all findings to scan_report.txt.
- Include timestamp, target URL, vulnerability type, severity, affected URL, and payload in each report entry.

Error Handling:

- Wrap all requests calls in try/except blocks to handle connection timeouts and errors.
- Return appropriate messages when the target URL is unreachable or returns no forms.

DATA STORAGE :

Unlike database-driven applications, the scanner uses a simple file-based reporting system. Every scan automatically generates or overwrites scan_report.txt with structured findings.

Report File Structure:

- **Target URL:** The URL that was scanned.
- **Scan Timestamp:** Date and time the scan was performed.
- **Total Issues:** Count of all vulnerabilities discovered.
- **Vulnerability Type:** XSS or SQL Injection.
- **Severity:** HIGH (XSS) or CRITICAL (SQLi).
- **Affected URL:** The specific form action URL that is vulnerable.
- **Payload Used:** The exact payload that triggered the vulnerability.

Sample Report Output:

```
============================================================
  WEB APPLICATION VULNERABILITY SCAN REPORT
============================================================
Target URL : http://testphp.vulnweb.com
Scan Time  : 2025-03-23 14:35:12
Total Issues: 4
------------------------------------------------------------
[1] Type     : XSS
    Severity : High
    URL      : http://testphp.vulnweb.com/search.php
    Payload  : <script>alert('XSS')</script>
[2] Type     : SQLi
    Severity : Critical
    URL      : http://testphp.vulnweb.com/login.php
    Payload  : ' OR 1=1--
```

FRONTEND DEVELOPMENT:

https://web-scan--nithishnidumolu.replit.app

1. Setup Flask Templates:

Building the Web Application Vulnerability Scanner frontend involves a straightforward development process. The foundation is built using Flask's Jinja2 templating system. This includes creating the template structure, adding HTML/CSS styling, and organizing the project files.

- Create the templates/ directory.
- Add index.html (home page) and results.html (scan results page).

2. Design UI Components:

Reusable HTML components are built for all interactive elements — the URL input form, the scan button, the results table, and the severity badges. A dark security-themed stylesheet (style.css) defines the overall look and feel. Navigation between the home and results pages is handled by Flask routing.

3. Implement Frontend Logic:

In the final stage of frontend development, the HTML forms are connected to Flask routes via POST requests using standard HTML form submission. This includes:

- Connecting the URL input form to the /scan POST route.
- Rendering results dynamically using Jinja2 {{ }} template variables.

PROJECT EXECUTION.

Steps for Project Execution:

Step 1: Clone or Download the Project:

- a) Clone from GitHub or open directly in Replit.
- b) Ensure all files are present: app.py, scanner.py, templates/, static/

Step 2: Install Dependencies:

- a) Open a terminal and navigate to the project folder.
- b) Install all required packages:

```
pip install flask requests beautifulsoup4
```

Step 3: Configure Environment:

- a) Ensure Python 3.8+ is installed.
- b) Verify Flask is installed by running:

```
flask --version
```

Step 4: Start the Flask Server:

- Inside the project folder, run:

```
python app.py
```

http://localhost:5000

Output Screenshots :

- **Home Page / Landing Page :** Displays the scanner title and description with a URL input field and "Start Scan" button. Serves as the entry point for all users.

- **Scan In Progress :** After clicking Start Scan, the page shows a loading state while the backend fetches forms, injects payloads, and analyzes responses for the target URL.

- **Results Page — Vulnerabilities Found :** Displays all vulnerabilities detected with Type, Severity (color-coded badge), Affected URL, and the Payload that triggered it. Critical findings appear in red, High in orange.

- **Results Page — No Vulnerabilities :** If no issues are found, the page shows a green success message: 'No vulnerabilities detected on this target.' along with a 'Scan Another URL' button.

- **Generated Report (scan_report.txt) :** A structured text report saved automatically after each scan. Contains target URL, timestamp, total issue count, and detailed information for each vulnerability found.

- **Admin / Code View on Replit :** Shows the Replit IDE with the project files (app.py, scanner.py, templates/) and the live deployment panel confirming the app is running publicly.

Demo video link :

https://web-scan--nithishnidumolu.replit.app

Code / Replit link :

https://web-scan--nithishnidumolu.replit.app